Case Study: A Chronological Approach to Optimizing Kit End Date Prediction

Phase 1: Project Inception and Problem Formulation

The project initiated with the objective of improving the accuracy of predicting the kit end date for manufacturing orders. It was recognized early on that predicting an absolute timestamp can be difficult for predictive models to generalize. Therefore, the problem was formulated as a regression task aimed at predicting the continuous duration in hours between the kit start and kit end dates. By predicting the lead time, this duration could be dynamically added to any given start date to find the predicted end date. To align with operational needs such as Service Level Agreement (SLA) tracking, it was decided that this continuous prediction would subsequently be translated into categorical classifications defined by fixed numerical thresholds, specifically: Fast (under 24 hours), Normal (24 to 48 hours), and Slow (over 48 hours).

Phase 2: Initial Data Exploration and Nuance Discovery

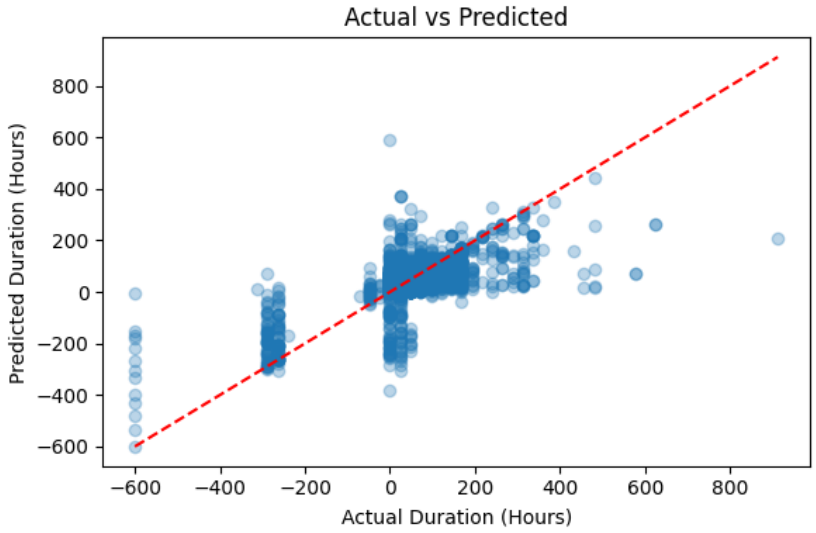
Following the problem formulation, the exploration phase commenced with an analysis of the schemas and summaries of five distinct datasets: `train.parquet` (179,853 records), `inference.parquet` (12,069 records), `calendar.parquet` (370 records), `capacity.parquet` (191,934 records), and `expected_minutes.parquet` (260 records). The primary goal of this stage was to understand the statistical nuances of the data before applying any transformations. Using Pearson correlation analysis on the numeric features via the `pandas` library, a critical discovery was made: the features `line_qty` and `quantity_produced` exhibited a highly positive correlation coefficient exceeding 0.93. It was determined that including both would introduce severe multicollinearity, potentially destabilizing model coefficients and feature importance scores. Additionally, it was observed that several crucial numerical fields were initially stored as strings or objects, requiring typecasting. Finally, the duration data was found to be heavily concentrated around specific hour marks, such as exactly 24 or 48 hours. This statistical nuance directly informed the subsequent decision to manually set fixed bins rather than relying on standard statistical quantiles for classification.

Phase 3: Data Engineering and Feature Synthesis

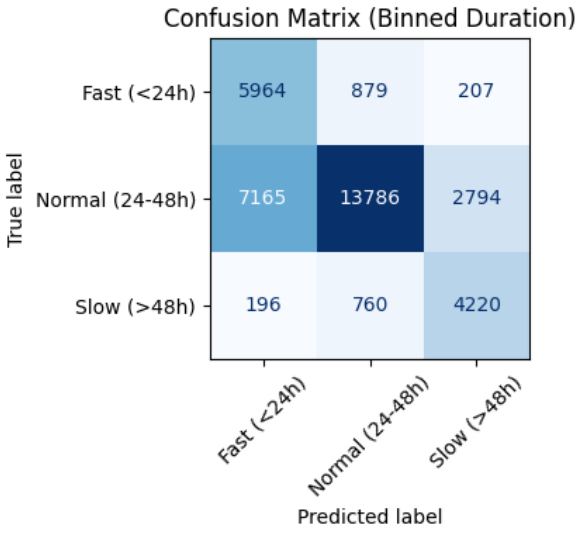
Once the data nuances were understood, the engineering phase was executed to transform the raw data into a cohesive, enriched modeling dataset comprising 179,853 rows and 43 columns. First, the continuous target variable was computed using vectorized datetime subtraction utilizing the `dt.total_seconds()` method in `pandas`. Next, quantities were coerced from object types into numeric floats via the `pd.to_numeric` function to enable mathematical operations. With the base data formatted, the tables were strategically joined using the `merge` function with `how='left'` arguments to build a rich feature space. The `expected_minutes.parquet` dataset was joined to incorporate baseline engineering standards, followed by the `calendar.parquet` dataset to inject temporal awareness such as day of the week, weekends, and holidays. Furthermore, the daily aggregated `capacity.parquet` dataset was incorporated via the `groupby` and `sum` aggregation methods to include labor availability metrics. To handle missing information, values were filled using median imputation utilizing the `fillna` function for numerical columns to ensure extreme outliers did not skew the imputed data. Any lingering missing values after the merges were zero-filled to satisfy the strict input requirements of the intended machine learning algorithms. Finally, the `line_qty` feature was dropped to resolve the high Pearson correlation identified during the earlier exploration phase.

Phase 4: Predictive Modeling and Performance Assessment

With the dataset fully engineered, the predictive modeling phase was initiated. Two robust ensemble tree-based algorithms from the `sklearn.ensemble` module, specifically `RandomForestRegressor` and `GradientBoostingRegressor`, were selected and evaluated using an 80/20 partition generated by `train_test_split`. These statistical models were chosen because they handle non-linear relationships well and do not require extensive feature scaling. The assessment of regression metrics, calculated using `numpy.sqrt` and `sklearn.metrics.mean_squared_error`, revealed that the Random Forest algorithm significantly outperformed Gradient Boosting, achieving a lower Root Mean Squared Error (RMSE) of 26.09 compared to 32.60, and a substantially higher R-squared value of 0.5241 versus 0.2567.



To ensure the predictions provided actionable business value, the continuous regression outputs were then discretely binned utilizing the `pandas.cut` function into the previously defined operational categories: Fast, Normal, and Slow. A classification confusion matrix was generated utilizing `sklearn.metrics.confusion_ma` to assess how well the continuous regression mapped to these practical SLA buckets, allowing stakeholders to easily evaluate misclassification rates across the 35,971 test samples.

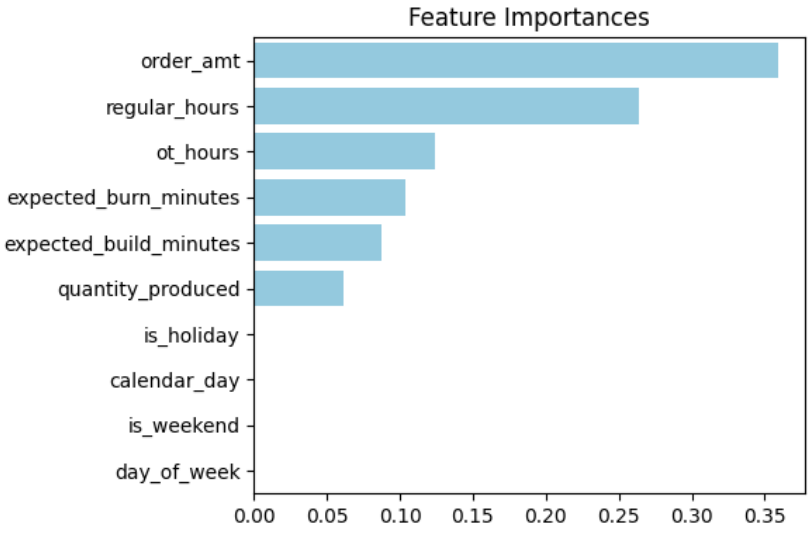


Phase 5: Code Architecture and Pipeline Design

Throughout the chronological execution of these steps, the codebase was developed modularly using standard data science libraries, notably `pandas`, `numpy`, `scikit-learn`, `matplotlib`, and `seaborn`. Vectorized operations were implemented for mathematical and datetime arithmetic rather than relying on slower iterative loops to ensure algorithmic efficiency. Memory efficiency was prioritized by utilizing left joins to prevent row explosion during the dataset merges. Visualizations were constructed leveraging `matplotlib.pyplot.subplots` for figure structuring, `seaborn.barplot` for feature importances, and `sklearn.metrics.ConfusionMatrixDisplay` for classification evaluation. Furthermore, inline comments and logical step-by-step blocking for transformations, joins, correlations, modeling, and plotting were consistently utilized. This architectural decision ensured that the pipeline remained reproducible, readable, and easily transferable to other engineers.

Phase 6: Strategic Presentation and Actionable Insights

Ultimately, the project culminated in the delivery of actionable insights and strategic recommendations based on the model's findings. An analysis of the Random Forest `feature_importances_` attribute revealed that the primary drivers of kit duration were order amount (importance score of ∼0.36), labor capacity through regular hours (∼0.26) and overtime hours (∼0.12), and quantity produced (∼0.06).



The statistical implication is that the physical volume of the order largely dictates the timeline, while the available workforce directly impacts throughput. Consequently, it was recommended that the business implement a dynamic scheduling tool to automatically flag incoming orders with high quantities produced and low available regular hours as high-risk for slipping into the Slow SLA category. Furthermore, the model's predicted duration can now be used to proactively approve overtime hours. If the model predicts an order will take over 48 hours but the customer SLA requires 24 hours, operations can immediately assign overtime to that specific site. Finally, while the expected build minutes feature was influential, it was not the sole driver. This insight prompted the recommendation that the business investigate discrepancies between expected engineering standards and actual durations, adjusting standard routing times accordingly to optimize future manufacturing planning.